UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Smart Resource Allocation System

Submitted by:

Team Members

NAME	SRN
Lakshita Negi	PES2UG23CS301
Kshirin Shetty	PES2UG23CS289

6-E

Prof Nandhi Kesavan

January - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

Problem Statement:

Modern computing systems often struggle with efficiently managing and allocating limited resources such as CPU and memory when multiple users submit tasks simultaneously. Existing approaches lack intelligent scheduling, leading to issues like resource underutilization, task delays, and unfair execution order. Additionally, there is limited support for dynamic configuration, real-time monitoring, and proper lifecycle management of tasks. This creates a need for a smart, scalable system that can efficiently schedule tasks, allocate resources based on availability and priority, and provide transparency and control to both users and administrators.

Key Features:

• Smart scheduling using FCFS, Priority-based, and Shortest Job First strategies.

• Dynamic allocation and release of CPU and memory based on availability.

• Complete task lifecycle management (Pending, Running, Completed, Cancelled).

• User operations: submit, edit, cancel, and track task status.

• Admin controls for configuring resource pool and managing scheduler.

• Real-time monitoring dashboard for system activity and resource utilization.

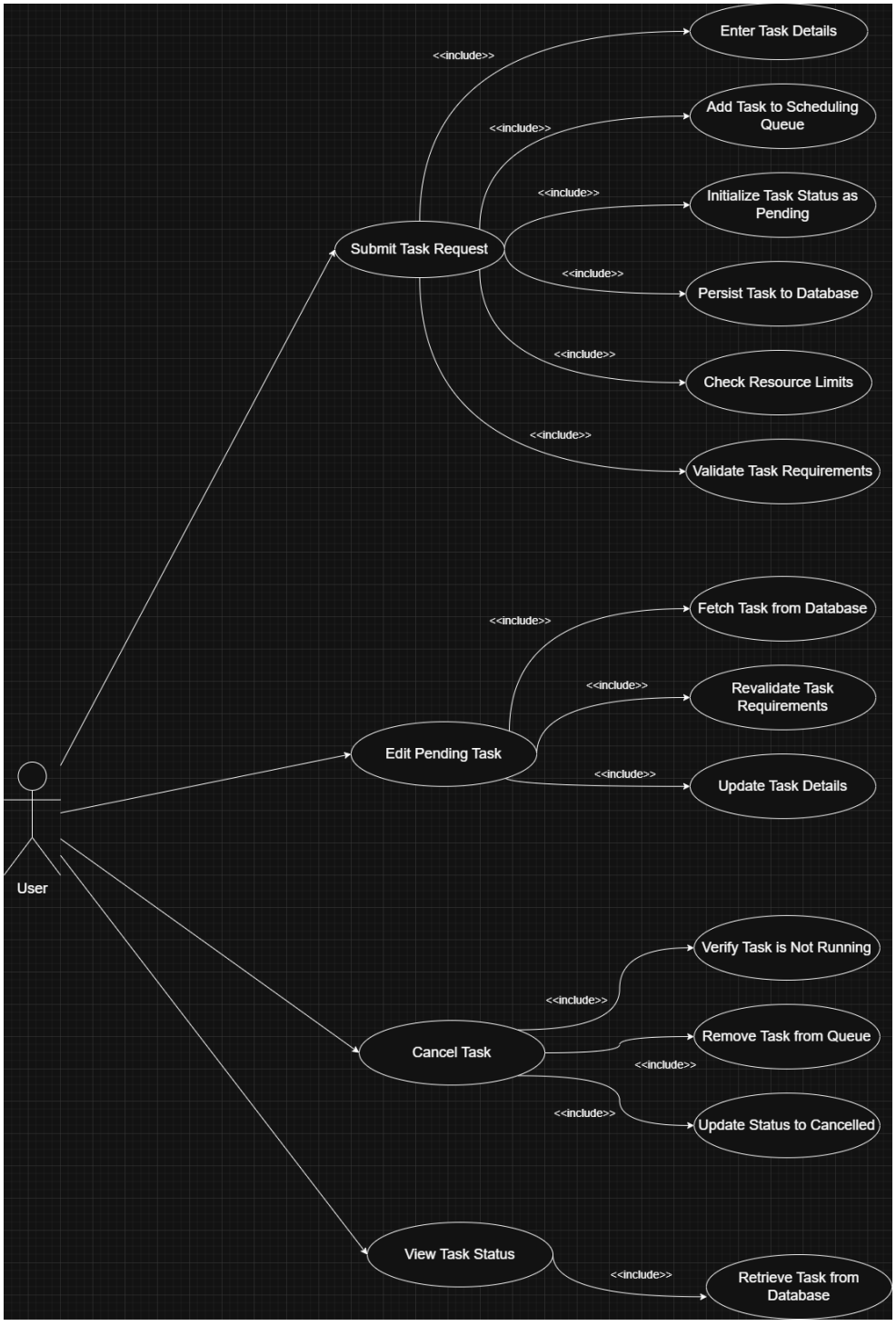
• Logging mechanism for allocation and completion events.

• Validation of task requirements and resource constraints before execution.

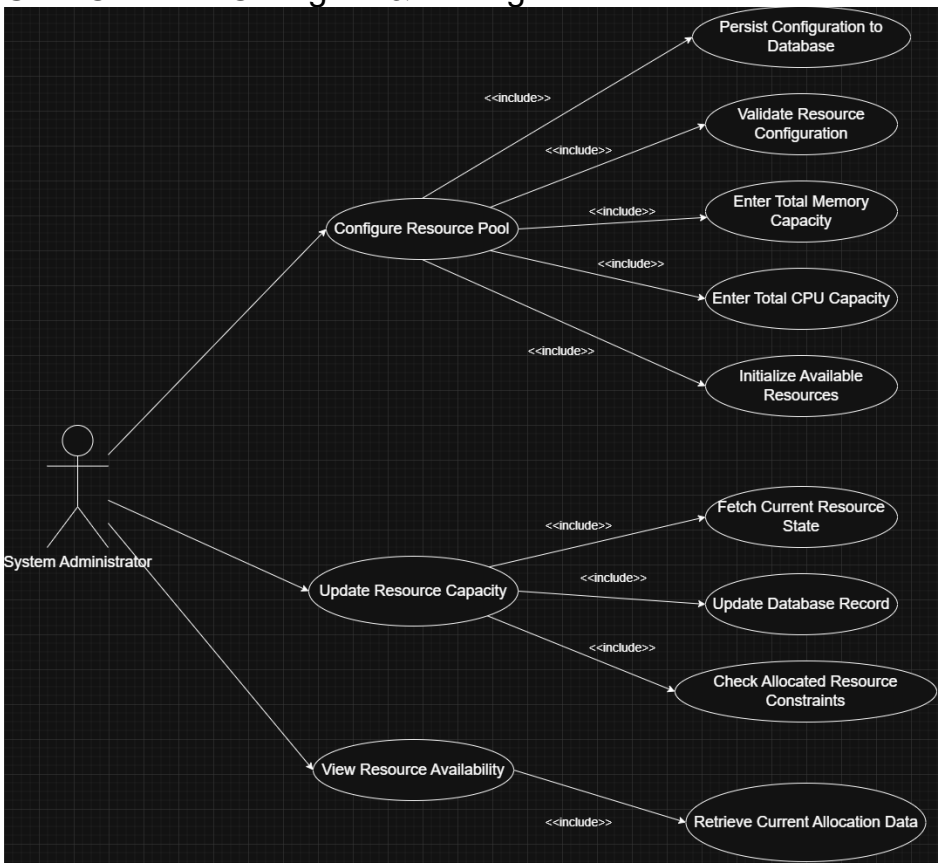
MODELS:

Use Case Diagram:

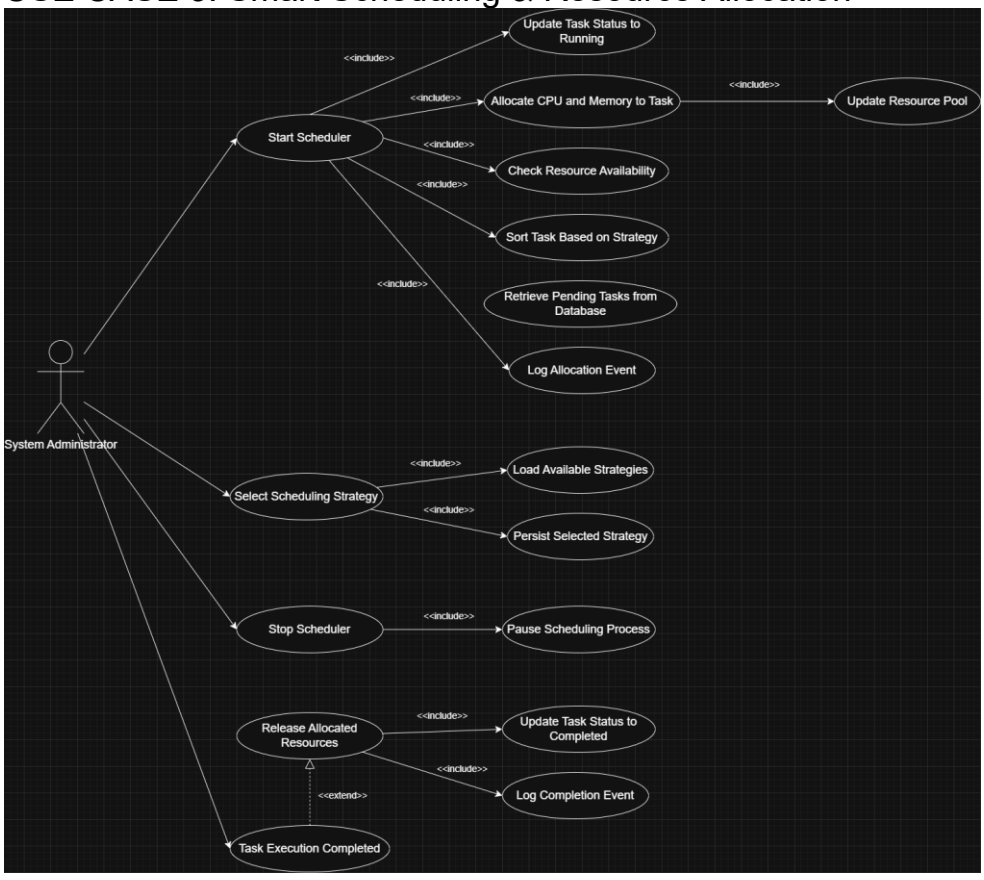
USE CASE 1: Submit & Manage Task Requests



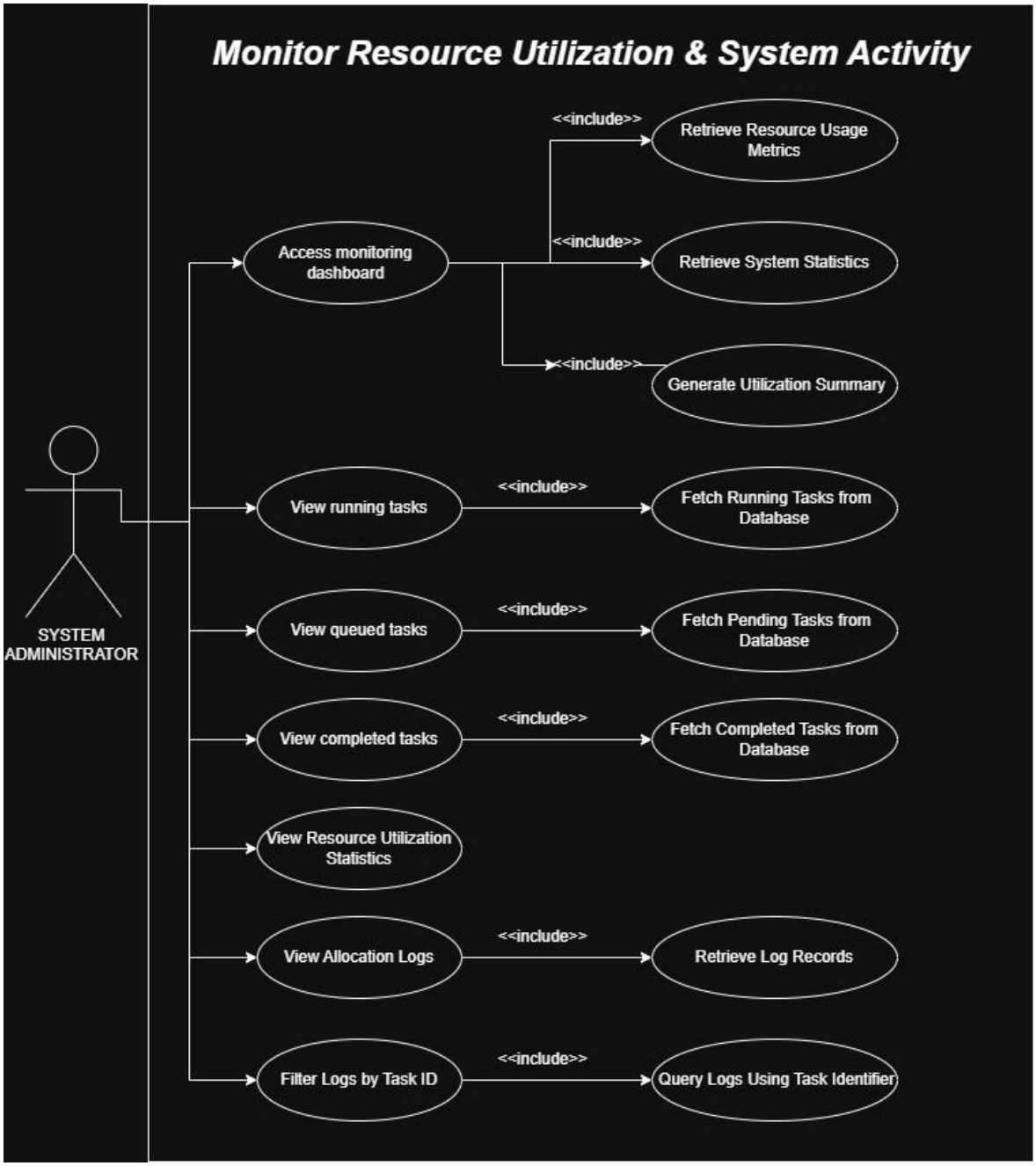
USE CASE 2: Configure & Manage Resource Pool



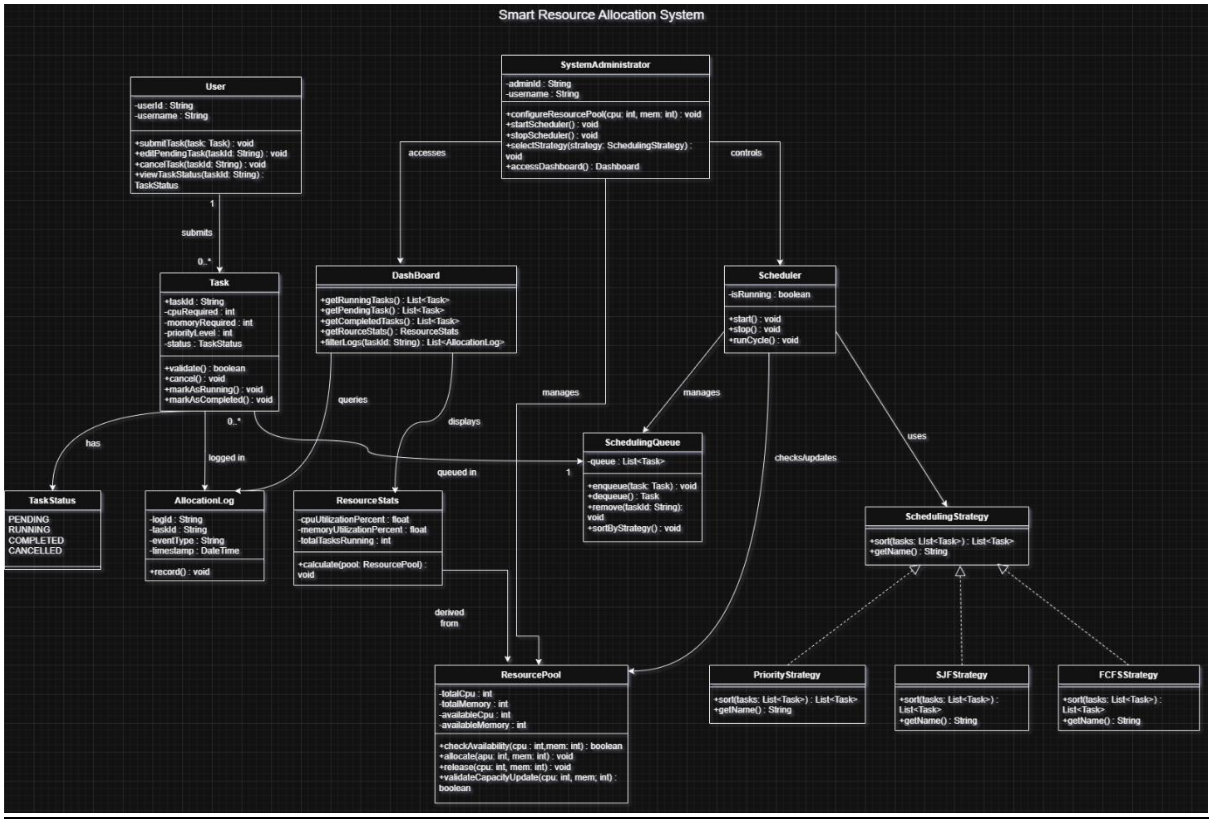
USE CASE 3: Smart Scheduling & Resource Allocation



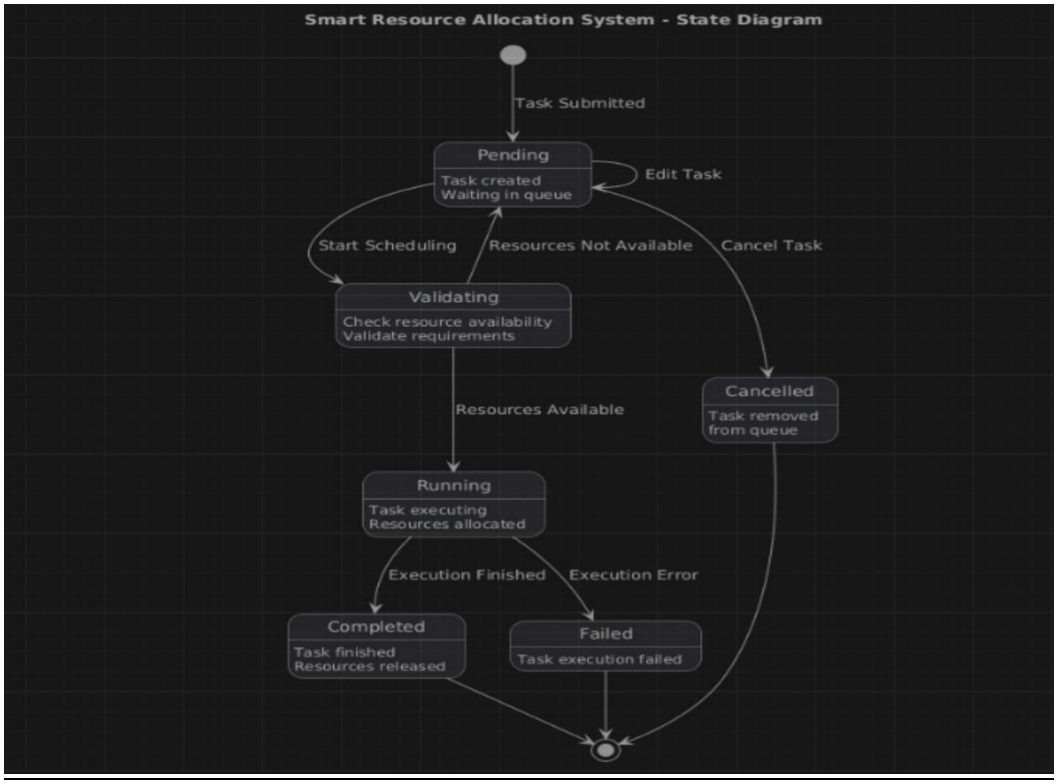
USE CASE 4: Monitor Resource Utilization & System Activity



Class Diagram:

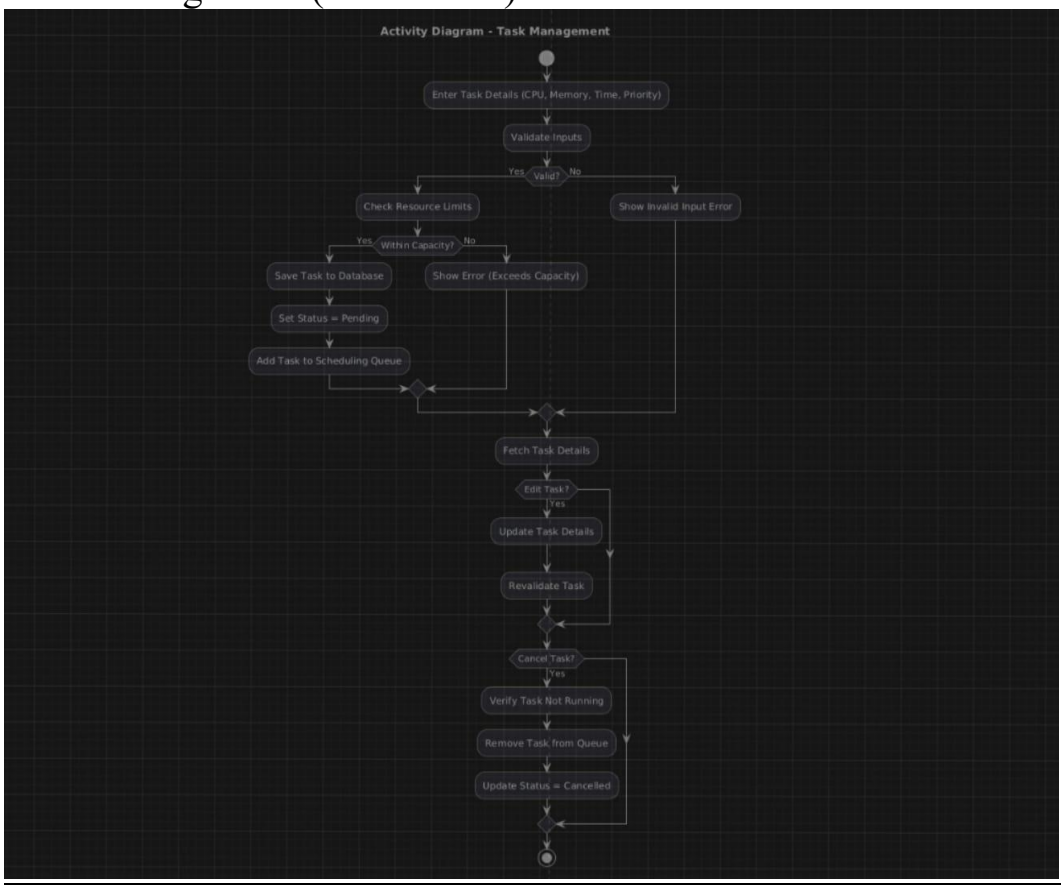


State Diagram:

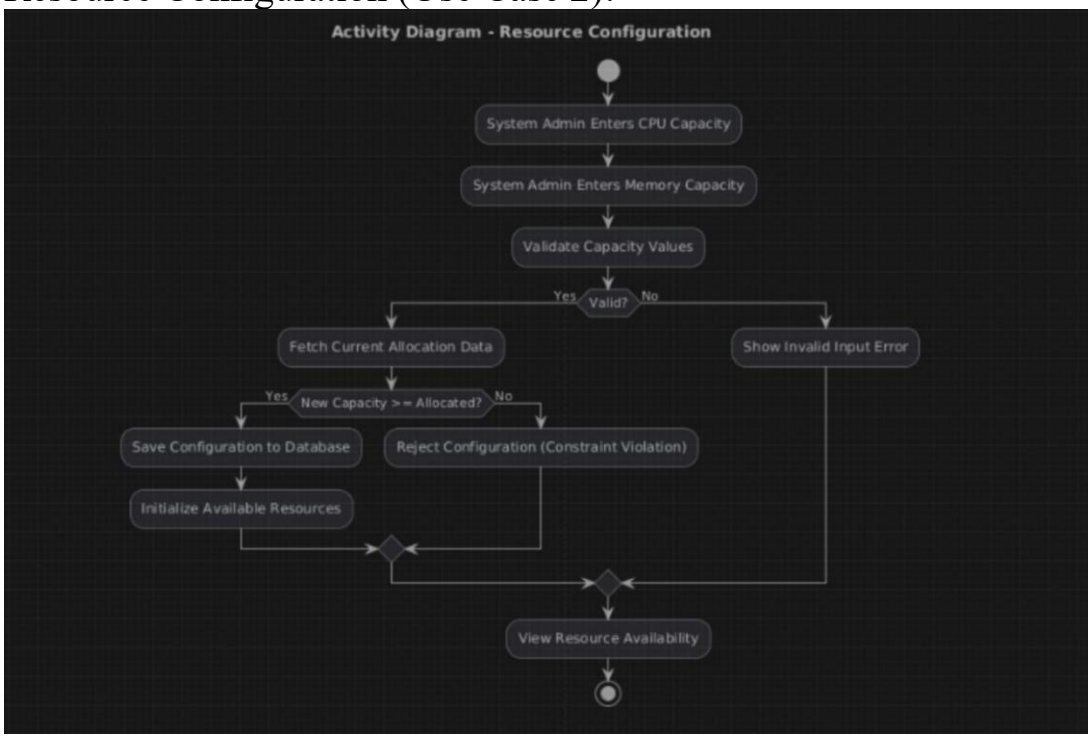


Activity Diagram:

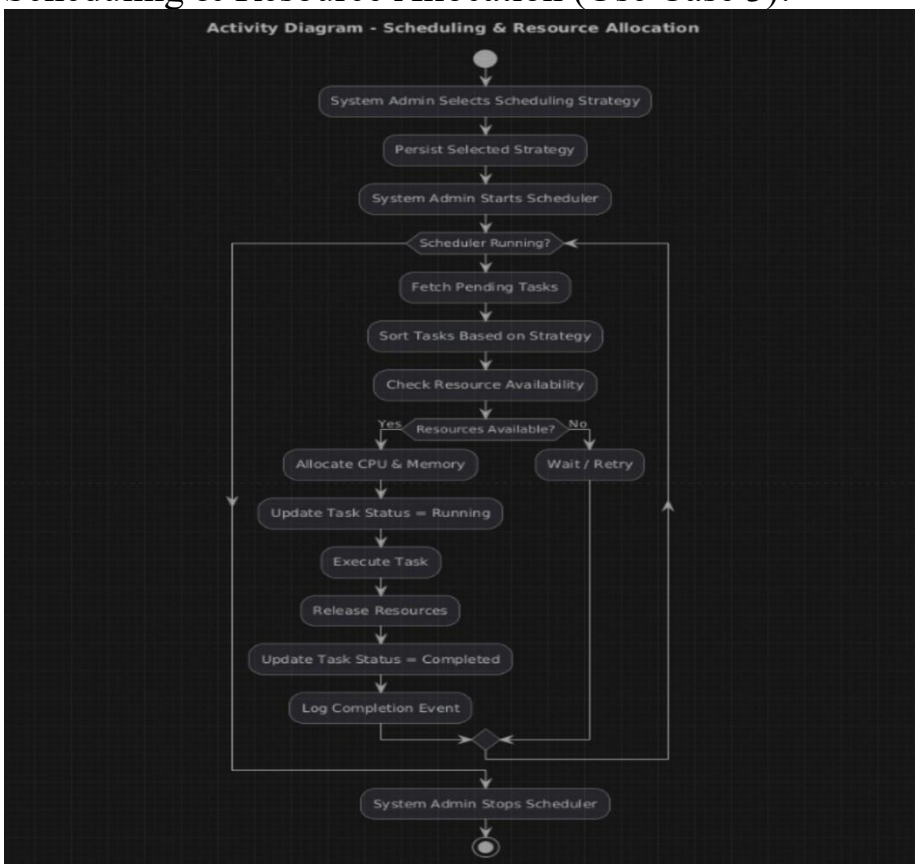
Task Management (Use Case 1):



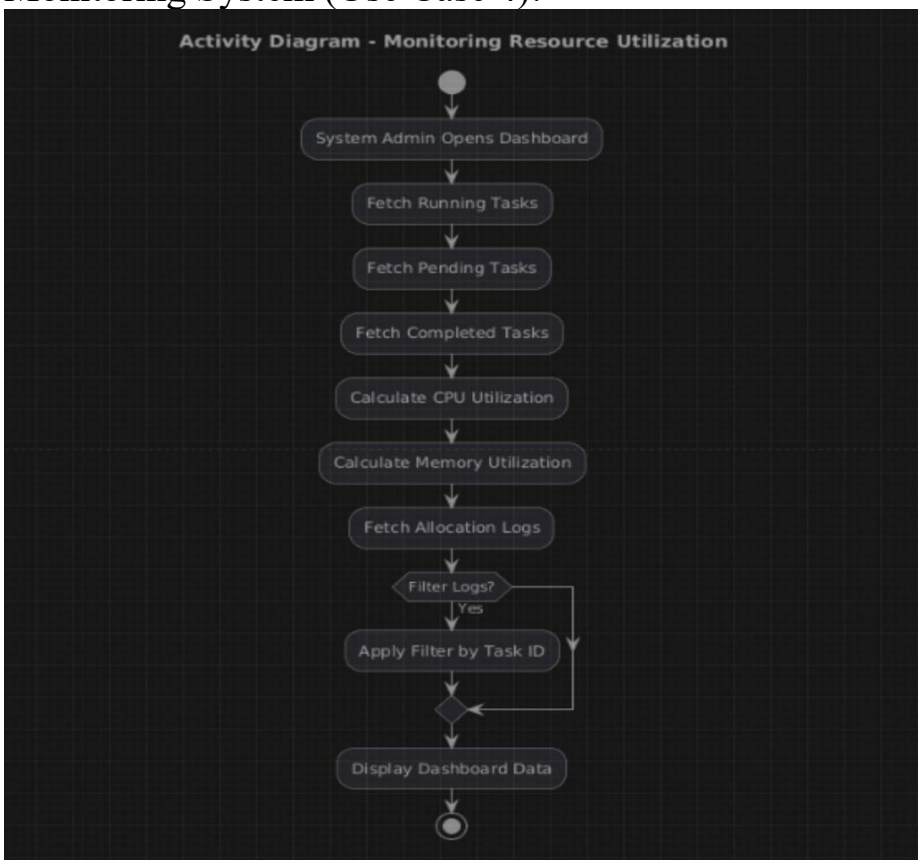
Resource Configuration (Use Case 2):



Scheduling & Resource Allocation (Use Case 3):



Monitoring System (Use Case 4):



MVC Architecture Explanation:

• Model: Task, ResourcePool, Scheduling components manage data and logic
• View: UI dashboard displays tasks and resource usage
• Controller: Scheduler and System Administrator handle system operations and user actions

Design Principles

SOLID Principles:

• Single Responsibility Principle (SRP): Each class has a single, well-defined responsibility. For instance, Task manages task data and lifecycle, Scheduler handles execution logic, and ResourcePool manages resource allocation.

• Open-Closed Principle (OCP): The system supports extension without modifying existing code. New scheduling algorithms can be added by implementing the SchedulingStrategy interface.

• Liskov Substitution Principle (LSP): All scheduling strategies (FCFS, Priority, SJF) can be used interchangeably without affecting system correctness.

• Interface Segregation Principle (ISP): Interfaces such as SchedulingStrategy are minimal and specific, preventing unnecessary method implementation.

• Dependency Inversion Principle (DIP): High-level components like Scheduler depend on abstractions (SchedulingStrategy), not concrete implementations.

GRASP Principles:

• Information Expert: ResourcePool manages resource allocation, and Task manages its own state transitions.

• Creator: Scheduler is responsible for handling task execution and lifecycle progression.

• Low Coupling: Components like Scheduler, ResourcePool, and Database interact through well-defined interfaces.

• High Cohesion: Each class performs a focused function, improving readability

and maintainability.

• Controller: SystemAdministrator acts as the controller for operations like starting/stopping the scheduler and configuring resources.

• Polymorphism: Multiple scheduling strategies are implemented using a common interface.

• Pure Fabrication: Classes such as AllocationLog and Dashboard are introduced to handle logging and monitoring.

• Indirection: SchedulingStrategy acts as an intermediate layer between task selection and execution.

• Protected Variations: Changes in scheduling logic or resource handling do not affect other components due to abstraction.

Design Patterns

Creational Patterns:

• Singleton Pattern: ResourcePool is implemented as a singleton to maintain a single consistent source of resource data across the system.

• Factory Pattern (Simple Factory): Used to instantiate different SchedulingStrategy objects dynamically based on user/admin selection.
Structural Patterns:

Structural Patterns:

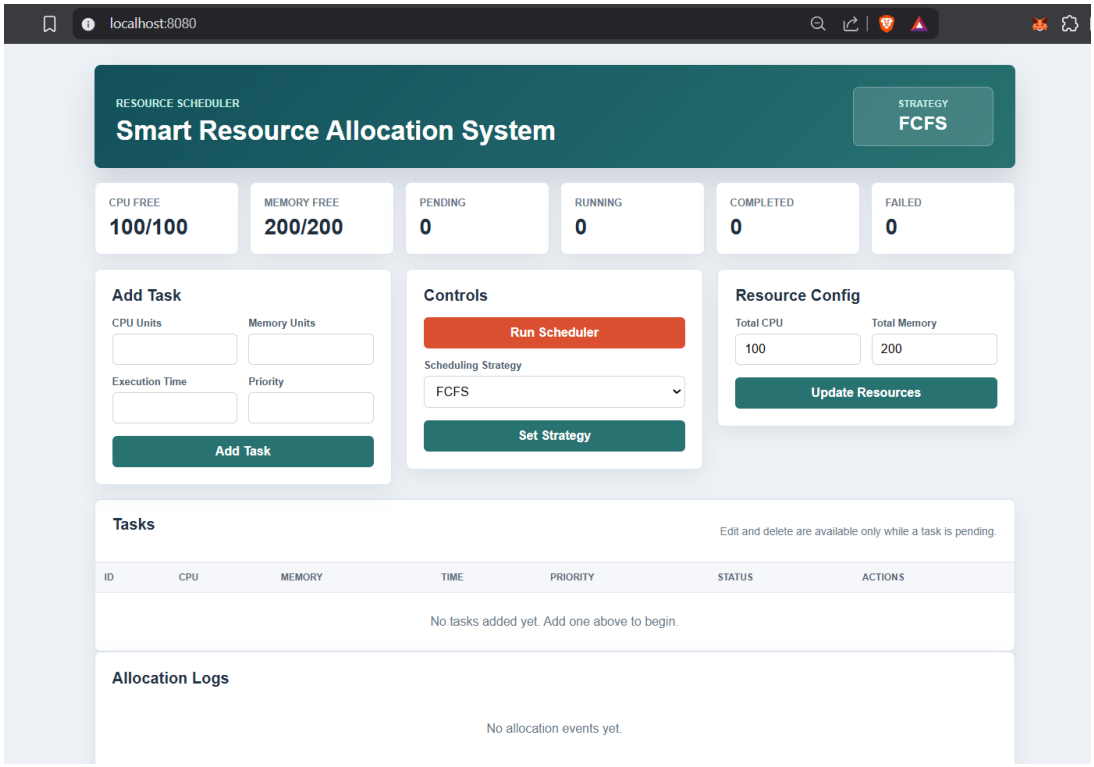
• Facade Pattern (Partial Implementation): The SchedulerService class provides a simplified interface for executing the scheduling process.
It abstracts the internal workflow, including fetching tasks, applying scheduling strategies, allocating resources, executing tasks, and releasing resources.

Behavioral Patterns:

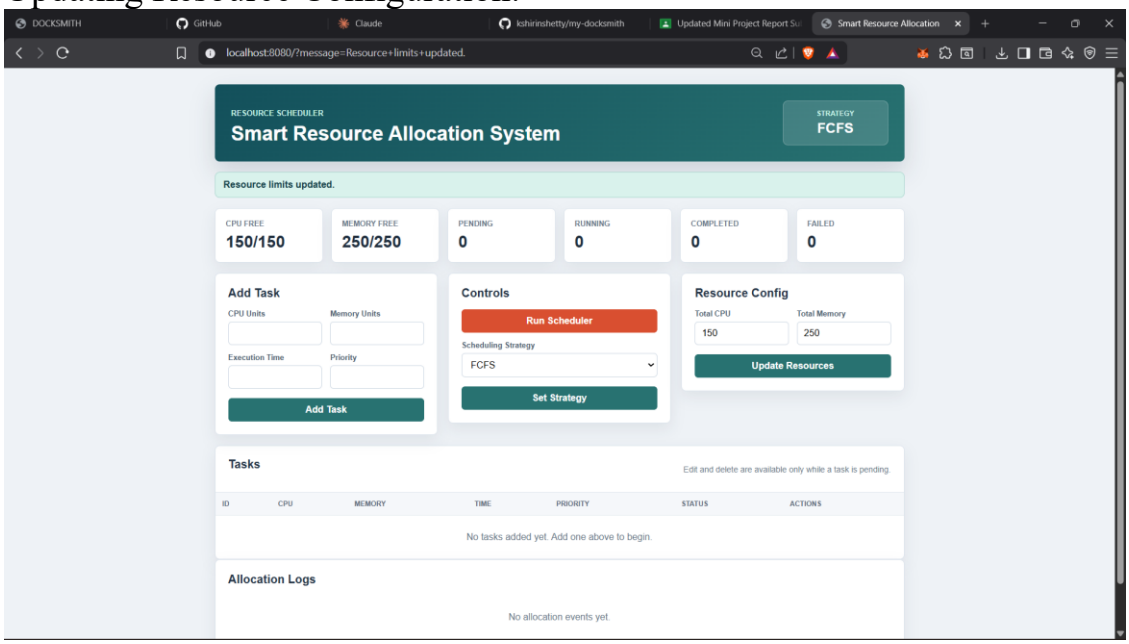
• Strategy Pattern: Core pattern used for scheduling. Different algorithms (FCFS, Priority, SJF) are encapsulated and can be switched dynamically at runtime.

• Iterator Pattern: The system uses Java's built-in iteration mechanisms such as enhanced for-loops and streams to traverse task collections for scheduling and display.

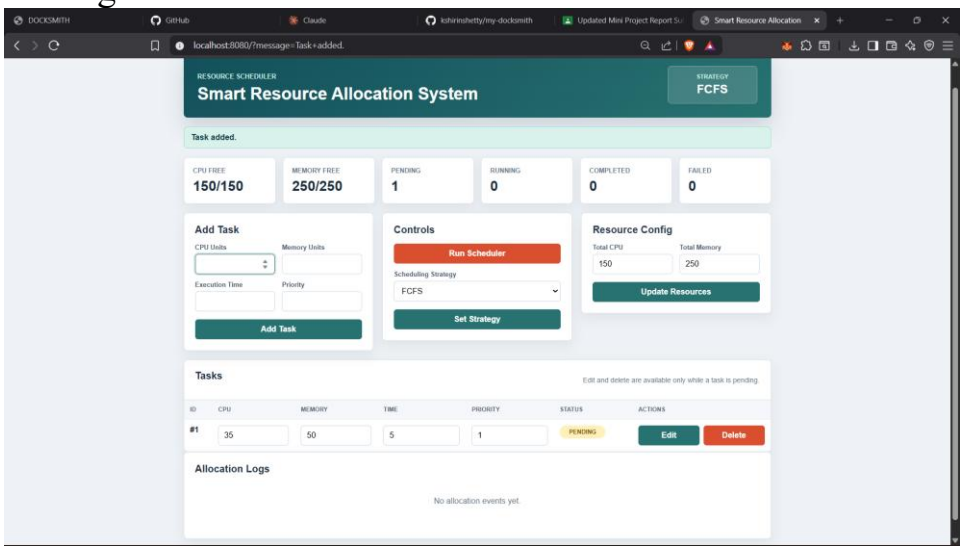
Screenshots of UI:



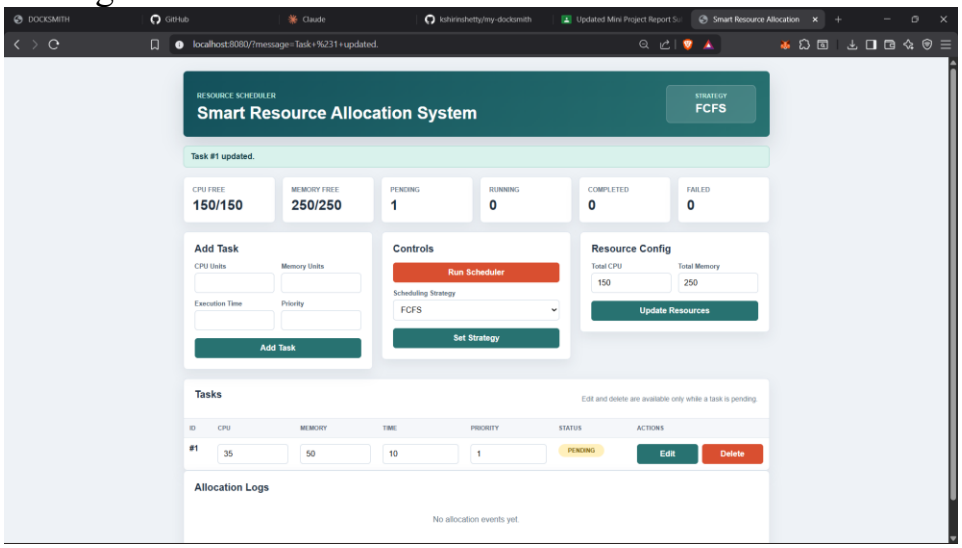
Updating Resource Configuration:



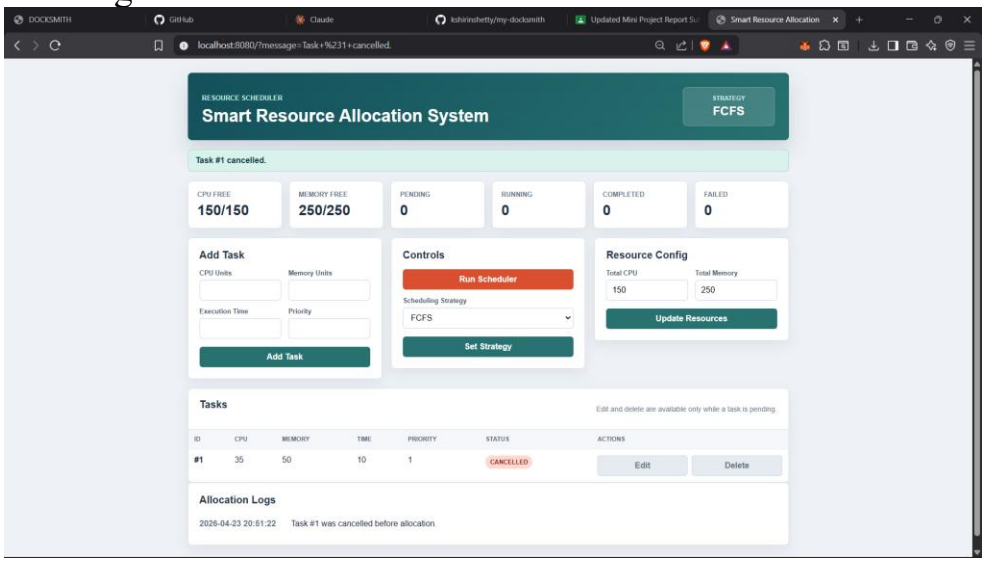
Adding Task:



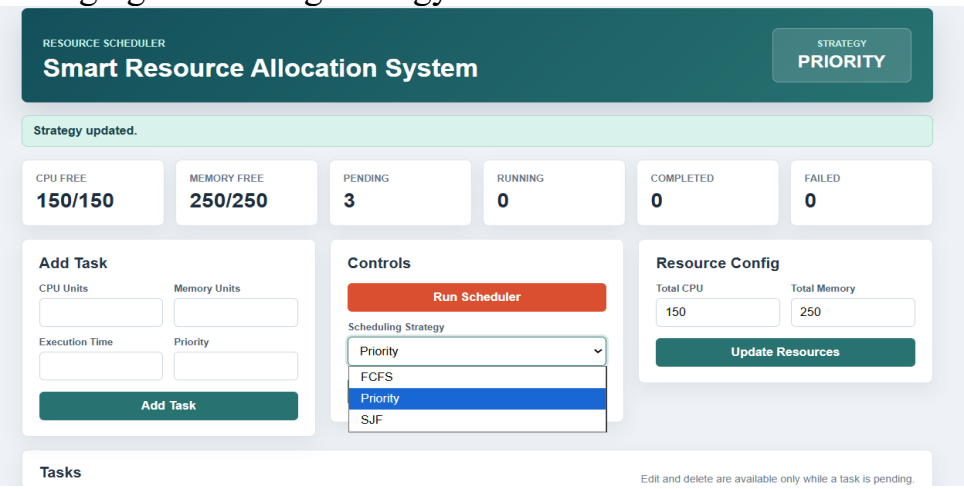
Editing Task:



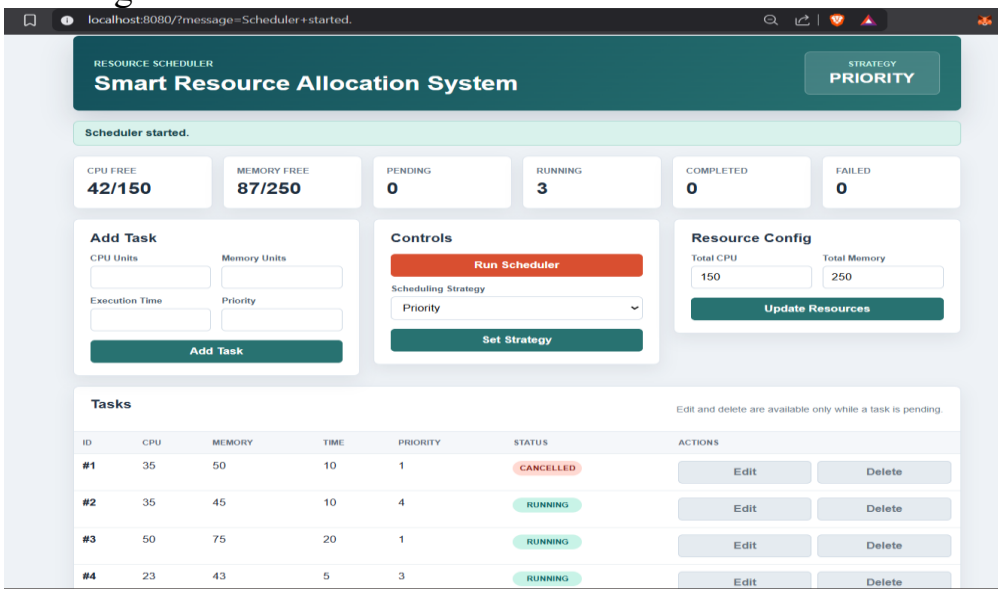
Deleting Task:



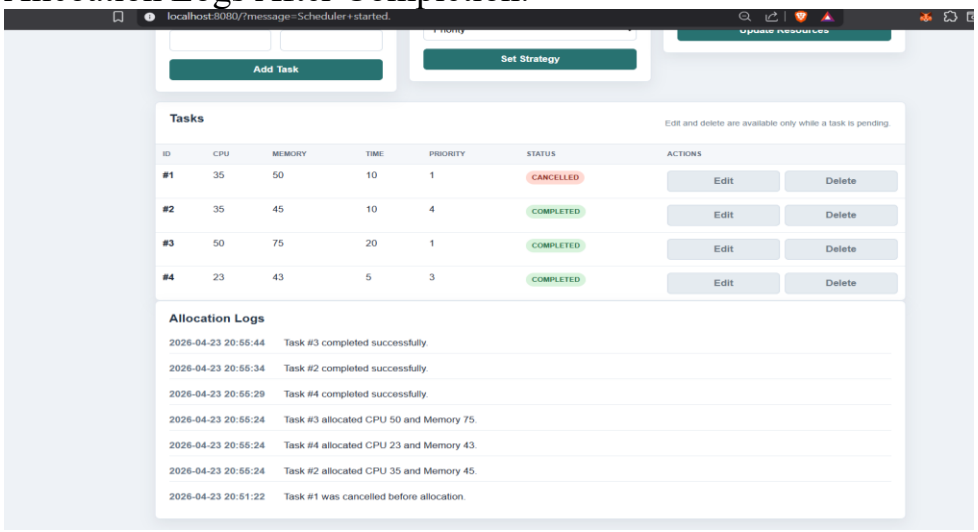
Changing Scheduling strategy:



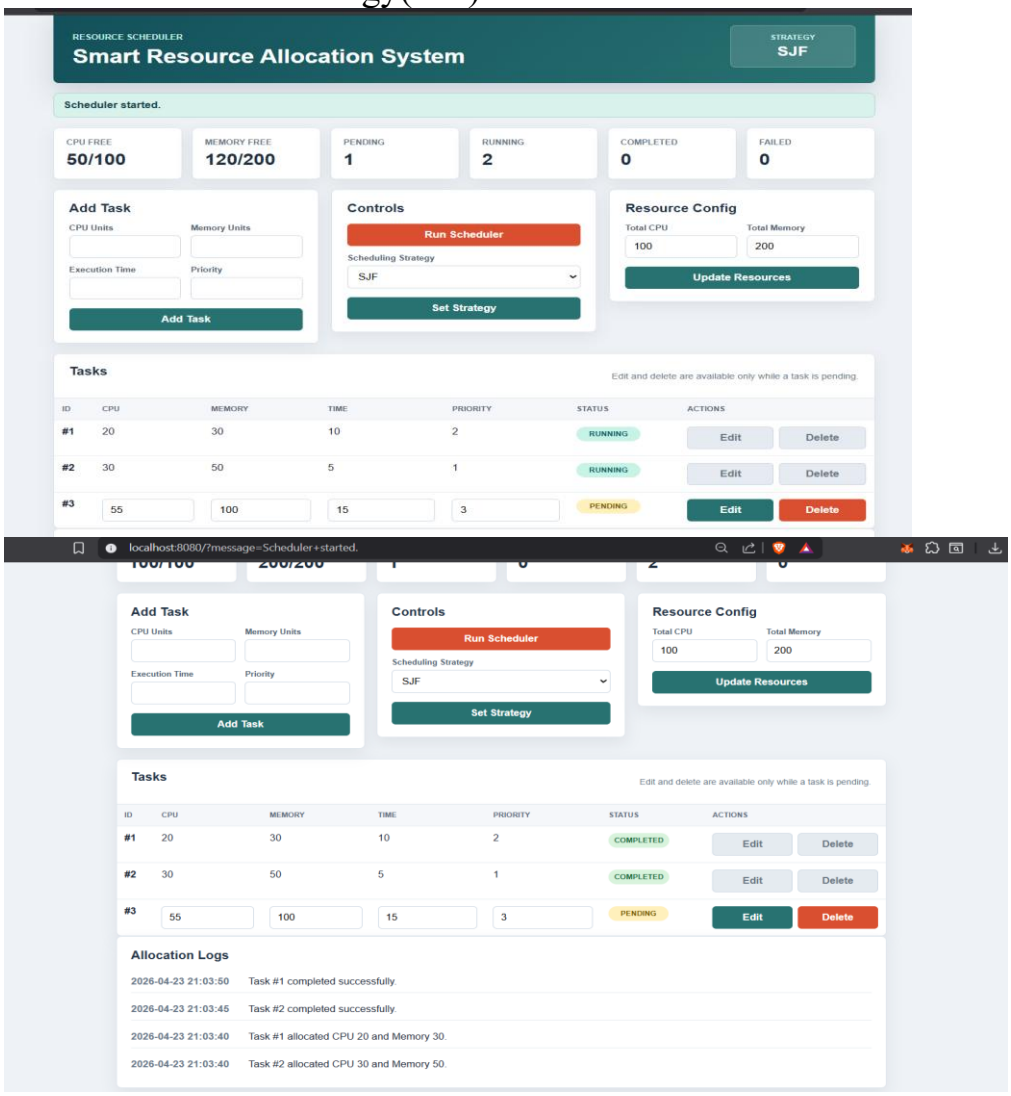
Running Scheduler:



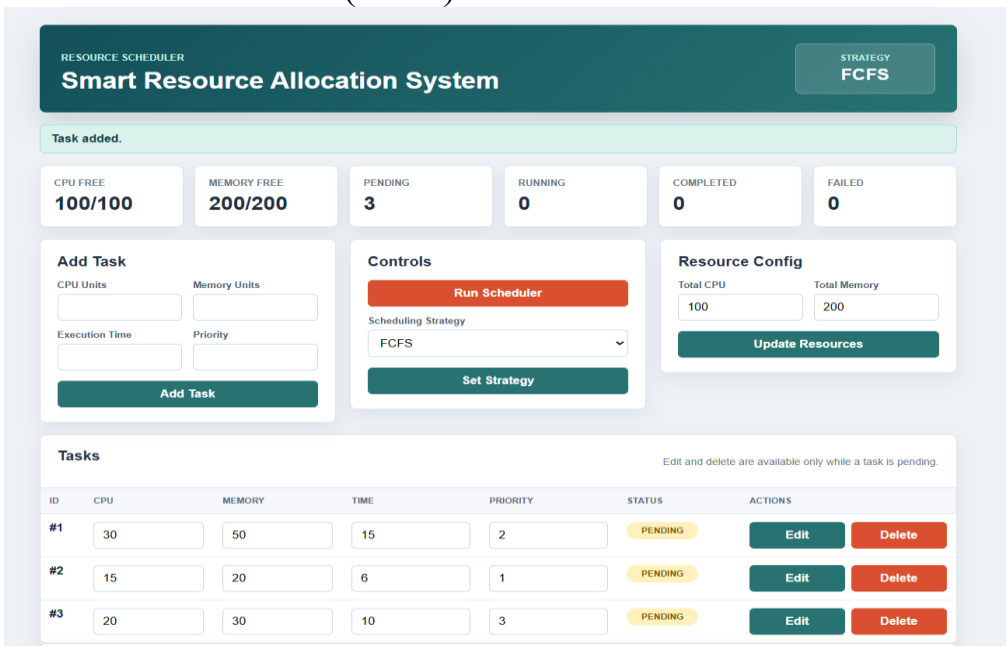
Allcoation Logs After Completion:

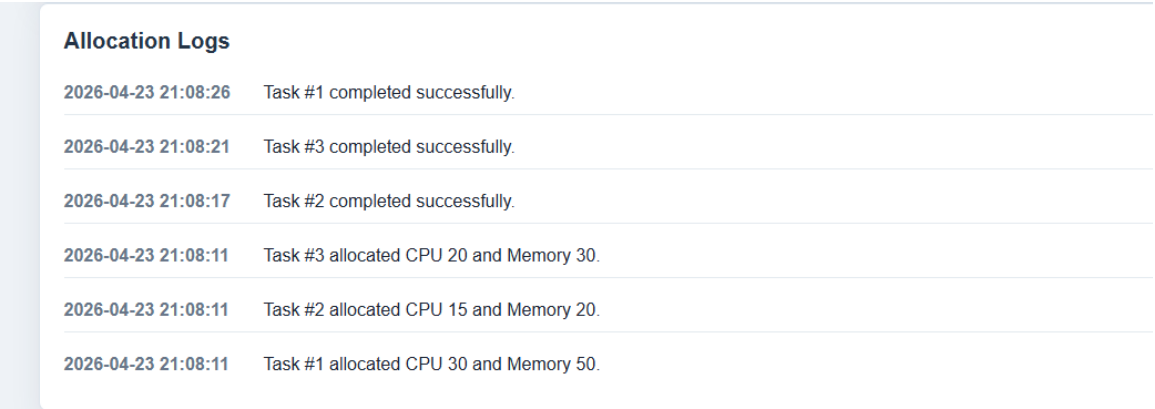


Shortest Job First Strategy(SJF):



First Come First Serve(FCFS):





Individual contributions of the team members:

Name	Module worked on
Lakshita Negi	• Task Management Module (Submit, Edit, Cancel, View Status) • Use Case Diagram (User-related) • Activity Diagrams (Task lifecycle flows) • Frontend UI (Task input, display, interactions) • Integration of Task module with Scheduler
Kshirin Shetty	• Resource Management & Scheduler Module • Use Case Diagram (Admin-related) • Activity Diagrams (Scheduling, Resource Allocation, Monitoring) • Implementation of Scheduling Strategies (FCFS, Priority, SJF) • Dashboard & Monitoring + Resource Pool Logic